

My Project

Generated by Doxygen 1.13.2

1 MSDScript	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Class Documentation	9
5.1 Add Class Reference	9
5.1.1 Detailed Description	10
5.1.2 Constructor & Destructor Documentation	10
5.1.2.1 Add()	10
5.1.3 Member Function Documentation	10
5.1.3.1 equals()	10
5.1.3.2 has_variable()	11
5.1.3.3 interp()	11
5.1.3.4 prettyPrint()	11
5.1.3.5 prettyPrintAt()	11
5.1.3.6 print()	11
5.1.3.7 subst()	12
5.1.4 Member Data Documentation	12
5.1.4.1 lhs	12
5.2 Expr Class Reference	12
5.2.1 Detailed Description	13
5.2.2 Member Function Documentation	13
5.2.2.1 equals()	13
5.2.2.2 has_variable()	14
5.2.2.3 interp()	14
5.2.2.4 prettyPrint()	14
5.2.2.5 prettyPrintAt()	14
5.2.2.6 print()	14
5.2.2.7 subst()	15
5.2.2.8 to_pretty_string()	15
5.2.2.9 to_string()	15
5.3 Mult Class Reference	16
5.3.1 Detailed Description	17
5.3.2 Constructor & Destructor Documentation	17
5.3.2.1 Mult()	17
5.3.3 Member Function Documentation	17
5.3.3.1 equals()	17

5.3.3.2 has_variable()	17
5.3.3.3 interp()	18
5.3.3.4 prettyPrint()	18
5.3.3.5 prettyPrintAt()	18
5.3.3.6 print()	18
5.3.3.7 subst()	19
5.3.4 Member Data Documentation	19
5.3.4.1 lhs	19
5.4 Num Class Reference	19
5.4.1 Detailed Description	20
5.4.2 Constructor & Destructor Documentation	20
5.4.2.1 Num()	20
5.4.3 Member Function Documentation	21
5.4.3.1 equals()	21
5.4.3.2 has_variable()	22
5.4.3.3 interp()	22
5.4.3.4 print()	22
5.4.3.5 subst()	22
5.4.4 Member Data Documentation	23
5.4.4.1 val	23
5.5 Variable Class Reference	23
5.5.1 Detailed Description	24
5.5.2 Constructor & Destructor Documentation	24
5.5.2.1 Variable()	24
5.5.3 Member Function Documentation	24
5.5.3.1 equals()	24
5.5.3.2 has_variable()	25
5.5.3.3 interp()	25
5.5.3.4 print()	25
5.5.3.5 subst()	25
5.5.4 Member Data Documentation	26
5.5.4.1 name	26
6 File Documentation	27
6.1 /Users/jasonablott/msdscript/cmdline.h	27
6.2 /Users/jasonablott/msdscript/expr.h	27
Index	29

Chapter 1

MSDScript

Author

Jason Ablott

Date

04-02-2025

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Expr	12
Add	9
Mult	16
Num	19
Variable	23

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Add	Add class: a derived expression class representing an addition expression	9
Expr	Expr class: a class representing a generic expression. See details for expression grammar. An expression can be a number, variable, or combination of those with a mathematical operator * or +	12
Mult	Mult class: a derived expression class representing a multiplication expression	16
Num	Num class: a derived expression class representing a number expression	19
Variable	Variable class: a derived expression class representing a variable expression	23

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

/Users/jasonablott/msdscript/ cmdline.h	27
/Users/jasonablott/msdscript/ expr.h	27

Chapter 5

Class Documentation

5.1 Add Class Reference

[Add](#) class: a derived expression class representing an addition expression.

```
#include <expr.h>
```

Inheritance diagram for Add:



Public Member Functions

- [Add](#) ([Expr](#) *lhs, [Expr](#) *rhs)
[Add](#) expression constructor.
- bool [equals](#) ([Expr](#) *e)
determines equality between two [Add](#) expressions
- int [interp](#) ()
interprets an [Add](#) expression, evaluating the math contained within it
- bool [has_variable](#) ()
determines if an [Add](#) expression contains a variable
- [Expr](#) * [subst](#) (std::string s, [Expr](#) *e)
substitutes an expression for a given string
- void [print](#) (std::ostream &os)
prints this [Add](#) expression to an outputstream
- void [prettyPrint](#) (std::ostream &os)
prints this [Add](#) expression in a more readable format
- void [prettyPrintAt](#) (std::ostream &os, precedence_t precedence)
helper method to enable precedence accumulation for prettyPrint

Public Member Functions inherited from [Expr](#)

- virtual `~Expr()`=default
expression destructor
- `std::string to_string()`
allows use of print on an expression
- `std::string to_pretty_string()`
allows use of prettyPrint on an expression

Public Attributes

- [Expr](#) * `lhs`
An expression representing the left hand side of this [Add](#) expression.
- [Expr](#) * `rhs`
An expression representing the right hand side of this [Add](#) expression.

5.1.1 Detailed Description

[Add](#) class: a derived expression class representing an addition expression.

More details here:

5.1.2 Constructor & Destructor Documentation

5.1.2.1 Add()

```
Add::Add (
    Expr * lhs,
    Expr * rhs)
```

[Add](#) expression constructor.

FUNCTIONS

5.1.3 Member Function Documentation

5.1.3.1 equals()

```
bool Add::equals (
    Expr * e) [virtual]
```

determines equality between two [Add](#) expressions

Parameters

<code>e</code>	another Add expression that this Add expression is compared to
----------------	--

Returns

boolean, true for [Add](#) expressions that are equal, otherwise false

Implements [Expr](#).

5.1.3.2 has_variable()

```
bool Add::has_variable () [virtual]
```

determines if an [Add](#) expression contains a variable

Returns

boolean, true for [Add](#) expressions that contain any variables, otherwise false

Implements [Expr](#).

5.1.3.3 interp()

```
int Add::interp () [virtual]
```

interprets an [Add](#) expression, evaluating the math contained within it

Returns

int an integer representing the result of the evaluated [Add](#) expression

Implements [Expr](#).

5.1.3.4 prettyPrint()

```
void Add::prettyPrint (  
    std::ostream & os) [virtual]
```

prints this [Add](#) expression in a more readable format

Parameters

<i>os</i>	the outputstream for this Add expression to be printed to
-----------	---

Reimplemented from [Expr](#).

5.1.3.5 prettyPrintAt()

```
void Add::prettyPrintAt (  
    std::ostream & os,  
    precedence_t precedence) [virtual]
```

helper method to enable precedence accumulation for prettyPrint

Parameters

<i>os</i>	the output stream for this Add expression to be printed to
<i>precedence</i>	the accumulated precedence of this expression, used to determine parentheses

Reimplemented from [Expr](#).

5.1.3.6 print()

```
void Add::print (  
    std::ostream & os) [virtual]
```

prints this [Add](#) expression to an outputstream

Parameters

<code>os</code>	an output stream for this Add expression to be printed to
-----------------	---

Implements [Expr](#).

5.1.3.7 subst()

```
Expr * Add::subst (
    std::string s,
    Expr * e) [virtual]
```

substitutes an expression for a given string

Parameters

<code>s</code>	a string in this expression to be replaced with e
<code>e</code>	an expression to substitute into this expression for string s

Returns

Expr* a new [Add](#) expression with e substituted for occurrences of s

Implements [Expr](#).

5.1.4 Member Data Documentation**5.1.4.1 lhs**

```
Expr* Add::lhs
```

An expression representing the left hand side of this [Add](#) expression.

MEMBER VARIABLES

The documentation for this class was generated from the following files:

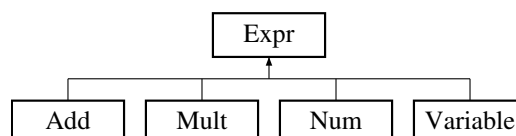
- /Users/jasonablott/msdscript/expr.h
- /Users/jasonablott/msdscript/expr.cpp

5.2 Expr Class Reference

[Expr](#) class: a class representing a generic expression. See details for expression grammar. An expression can be a number, variable, or combination of those with a mathematical operator * or +.

```
#include <expr.h>
```

Inheritance diagram for Expr:



Public Member Functions

- virtual `~Expr()`=default
expression destructor
- virtual bool `equals(Expr *e)`=0
determines equality between two expressions
- virtual int `interp()`=0
interprets an expression, evaluating the math contained within it
- virtual bool `has_variable()`=0
determines if an expression contains a variable
- virtual `Expr * subst(std::string s, Expr *e)`=0
substitutes an expression for a given string
- virtual void `print(std::ostream &os)`=0
prints this expression to an outputstream
- virtual void `prettyPrint(std::ostream &os)`
prints this expression in a more readable format
- virtual void `prettyPrintAt(std::ostream &os, precedence_t precedence)`
helper method to enable precedence accumulation for prettyPrint
- std::string `to_string()`
allows use of print on an expression
- std::string `to_pretty_string()`
allows use of prettyPrint on an expression

5.2.1 Detailed Description

`Expr` class: a class representing a generic expression. See details for expression grammar. An expression can be a number, variable, or combination of those with a mathematical operator `*` or `+`.

- The abstract grammar of `Expr` is: `<expr> = <number> | <expr> + <expr> | <expr> * <expr> | <variable>`

5.2.2 Member Function Documentation

5.2.2.1 `equals()`

```
virtual bool Expr::equals (
    Expr * e) [pure virtual]
```

determines equality between two expressions

Parameters

<code>e</code>	another expression that this expression is compared to
----------------	--

Returns

boolean, true for expressions that are equal, otherwise false

Implemented in `Add`, `Mult`, `Num`, and `Variable`.

5.2.2.2 has_variable()

```
virtual bool Expr::has_variable () [pure virtual]
```

determines if an expression contains a variable

Returns

boolean, true for expressions that contain any variables, otherwise false

Implemented in [Add](#), [Mult](#), [Num](#), and [Variable](#).

5.2.2.3 interp()

```
virtual int Expr::interp () [pure virtual]
```

interprets an expression, evaluating the math contained within it

Returns

int an integer representing the result of the evaluated expression

Implemented in [Add](#), [Mult](#), [Num](#), and [Variable](#).

5.2.2.4 prettyPrint()

```
void Expr::prettyPrint (
    std::ostream & os) [virtual]
```

prints this expression in a more readable format

Parameters

<i>os</i>	the outputstream for this expression to be printed to
-----------	---

Reimplemented in [Add](#), and [Mult](#).

5.2.2.5 prettyPrintAt()

```
void Expr::prettyPrintAt (
    std::ostream & os,
    precedence_t precedence) [virtual]
```

helper method to enable precedence accumulation for prettyPrint

Parameters

<i>os</i>	the output stream for this expression to be printed to
<i>precedence</i>	the accumulated precedence of this expression, used to determine parentheses

Reimplemented in [Add](#), and [Mult](#).

5.2.2.6 print()

```
void Expr::print (
    std::ostream & os) [pure virtual]
```

prints this expression to an outputstream

Parameters

<code>os</code>	an output stream for this expression to be printed to
-----------------	---

Implemented in [Add](#), [Mult](#), [Num](#), and [Variable](#).

5.2.2.7 subst()

```
virtual Expr * Expr::subst (
    std::string s,
    Expr * e) [pure virtual]
```

substitutes an expression for a given string

Parameters

<code>s</code>	a string in this expression to be replaced with e
<code>e</code>	an expression to substitute into this expression for string s

Returns

Expr* a new expression with e substituted for occurrences of s

Implemented in [Add](#), [Mult](#), [Num](#), and [Variable](#).

5.2.2.8 to_pretty_string()

```
std::string Expr::to_pretty_string ()
```

allows use of prettyPrint on an expression

Returns

string a string representing the expression

5.2.2.9 to_string()

```
std::string Expr::to_string ()
```

allows use of print on an expression

Returns

string a string representing the expression

Created by Jason Ablott on 1/20/25.

The documentation for this class was generated from the following files:

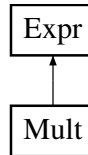
- `/Users/jasonablott/msdscript/expr.h`
- `/Users/jasonablott/msdscript/expr.cpp`

5.3 Mult Class Reference

Mult class: a derived expression class representing a multiplication expression.

```
#include <expr.h>
```

Inheritance diagram for Mult:



Public Member Functions

- **Mult** (**Expr** *lhs, **Expr** *rhs)
Mult expression constructor.
- bool **equals** (**Expr** *e)
determines equality between two Mult expressions
- int **interp** ()
interprets a Mult expression, evaluating the math contained within it
- bool **has_variable** ()
determines if a Mult expression contains a variable
- **Expr** * **subst** (std::string s, **Expr** *e)
substitutes an expression for a given string
- void **print** (std::ostream &os)
prints this Mult expression to an outputstream
- void **prettyPrint** (std::ostream &os)
prints this Mult expression in a more readable format
- void **prettyPrintAt** (std::ostream &os, precedence_t precedence)
helper method to enable precedence accumulation for prettyPrint

Public Member Functions inherited from Expr

- virtual ~**Expr** ()=default
expression destructor
- std::string **to_string** ()
allows use of print on an expression
- std::string **to_pretty_string** ()
allows use of prettyPrint on an expression

Public Attributes

- **Expr** * **lhs**
An expression representing the left hand side of this Mult expression.
- **Expr** * **rhs**
An expression representing the left hand side of this Mult expression.

5.3.1 Detailed Description

[Mult](#) class: a derived expression class representing a multiplication expression.

More details here:

5.3.2 Constructor & Destructor Documentation

5.3.2.1 Mult()

```
Mult::Mult (
    Expr * lhs,
    Expr * rhs)
```

[Mult](#) expression constructor.

FUNCTIONS

5.3.3 Member Function Documentation

5.3.3.1 equals()

```
bool Mult::equals (
    Expr * e) [virtual]
```

determines equality between two [Mult](#) expressions

Parameters

<i>e</i>	another Mult expression that this Mult expression is compared to
----------	--

Returns

boolean, true for expressions that are equal, otherwise false

Implements [Expr](#).

5.3.3.2 has_variable()

```
bool Mult::has_variable () [virtual]
```

determines if a [Mult](#) expression contains a variable

Returns

boolean, true for [Mult](#) expressions that contain any variables, otherwise false

Implements [Expr](#).

5.3.3.3 interp()

```
int Mult::interp () [virtual]
```

interprets a [Mult](#) expression, evaluating the math contained within it

Returns

int an integer representing the result of the evaluated [Mult](#) expression

Implements [Expr](#).

5.3.3.4 prettyPrint()

```
void Mult::prettyPrint (
    std::ostream & os) [virtual]
```

prints this [Mult](#) expression in a more readable format

Parameters

<i>os</i>	the outputstream for this Mult expression to be printed to
-----------	--

Reimplemented from [Expr](#).

5.3.3.5 prettyPrintAt()

```
void Mult::prettyPrintAt (
    std::ostream & os,
    precedence_t precedence) [virtual]
```

helper method to enable precedence accumulation for prettyPrint

Parameters

<i>os</i>	the output stream for this Mult expression to be printed to
<i>precedence</i>	the accumulated precedence of this expression, used to determine parentheses

Reimplemented from [Expr](#).

5.3.3.6 print()

```
void Mult::print (
    std::ostream & os) [virtual]
```

prints this [Mult](#) expression to an outputstream

Parameters

<code>os</code>	an output stream for this Mult expression to be printed to
-----------------	--

Implements [Expr](#).

5.3.3.7 subst()

```
Expr * Mult::subst (
    std::string s,
    Expr * e) [virtual]
```

substitutes an expression for a given string

Parameters

<code>s</code>	a string in this expression to be replaced with e
<code>e</code>	an expression to substitute into this expression for string s

Returns

Expr* a new [Mult](#) expression with e substituted for occurrences of s

Implements [Expr](#).

5.3.4 Member Data Documentation**5.3.4.1 lhs**

```
Expr* Mult::lhs
```

An expression representing the left hand side of this [Mult](#) expression.

MEMBER VARIABLES

The documentation for this class was generated from the following files:

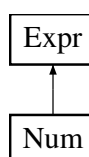
- /Users/jasonablott/msdscript/expr.h
- /Users/jasonablott/msdscript/expr.cpp

5.4 Num Class Reference

[Num](#) class: a derived expression class representing a number expression.

```
#include <expr.h>
```

Inheritance diagram for Num:



Public Member Functions

- [Num](#) (int [val](#))
Add expression constructor.
- bool [equals](#) ([Expr](#) *e)
determines equality between two [Num](#) expressions
- int [interp](#) ()
interprets a [Num](#) expression, evaluating the math contained within it
- bool [has_variable](#) ()
determines if a [Num](#) expression contains a variable
- [Expr](#) * [subst](#) (std::string s, [Expr](#) *e)
substitutes an expression for a given string
- void [print](#) (std::ostream &os)
prints this [Num](#) expression to an outputstream

Public Member Functions inherited from [Expr](#)

- virtual ~[Expr](#) ()=default
expression destructor
- virtual void [prettyPrint](#) (std::ostream &os)
prints this expression in a more readable format
- virtual void [prettyPrintAt](#) (std::ostream &os, precedence_t precedence)
helper method to enable precedence accumulation for prettyPrint
- std::string [to_string](#) ()
allows use of print on an expression
- std::string [to_pretty_string](#) ()
allows use of prettyPrint on an expression

Public Attributes

- int [val](#)
integer value representing this [Num](#) expression

5.4.1 Detailed Description

[Num](#) class: a derived expression class representing a number expression.

More details here:

5.4.2 Constructor & Destructor Documentation

5.4.2.1 Num()

```
Num::Num (
    int val)
```

[Add](#) expression constructor.

FUNCTIONS

5.4.3 Member Function Documentation

5.4.3.1 equals()

```
bool Num::equals (  
    Expr * e) [virtual]
```

determines equality between two [Num](#) expressions

Parameters

<code>e</code>	another Num expression that this Num expression is compared to
----------------	--

Returns

boolean, true for [Num](#) expressions that are equal, otherwise false

Implements [Expr](#).

5.4.3.2 has_variable()

```
bool Num::has_variable () [virtual]
```

determines if a [Num](#) expression contains a variable

Returns

boolean, always false as a number is not a variable

Implements [Expr](#).

5.4.3.3 interp()

```
int Num::interp () [virtual]
```

interprets a [Num](#) expression, evaluating the math contained within it

Returns

int an integer representing the result of the evaluated [Num](#) expression

Implements [Expr](#).

5.4.3.4 print()

```
void Num::print (
    std::ostream & os) [virtual]
```

prints this [Num](#) expression to an outputstream

Parameters

<code>os</code>	an output stream for this Num expression to be printed to
-----------------	---

Implements [Expr](#).

5.4.3.5 subst()

```
Expr * Num::subst (
    std::string s,
    Expr * e) [virtual]
```

substitutes an expression for a given string

Parameters

<i>s</i>	a string in this expression to be replaced with <i>e</i>
<i>e</i>	an expression to substitute into this expression for string <i>s</i>

Returns

Expr* a new [Num](#) expression with *e* substituted for occurrences of *s*

Implements [Expr](#).

5.4.4 Member Data Documentation

5.4.4.1 val

```
int Num::val
```

integer value representing this [Num](#) expression

MEMBER VARIABLES

The documentation for this class was generated from the following files:

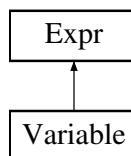
- /Users/jasonablott/msdscript/expr.h
- /Users/jasonablott/msdscript/expr.cpp

5.5 Variable Class Reference

[Variable](#) class: a derived expression class representing a variable expression.

```
#include <expr.h>
```

Inheritance diagram for Variable:



Public Member Functions

- [Variable](#) (std::string *name*)
Variable expression constructor.
- bool [equals](#) (Expr **e*)
determines equality between two Variable expressions
- int [interp](#) ()
interprets a Variable expression, evaluating the math contained within it
- bool [has_variable](#) ()
determines if a variable expression contains a variable
- Expr * [subst](#) (std::string *s*, Expr **e*)
substitutes an expression for a given string
- void [print](#) (std::ostream &*os*)
prints this Variable expression to an outputstream

Public Member Functions inherited from [Expr](#)

- virtual `~Expr()`=default
expression destructor
- virtual void `prettyPrint` (std::ostream &os)
prints this expression in a more readable format
- virtual void `prettyPrintAt` (std::ostream &os, precedence_t precedence)
helper method to enable precedence accumulation for prettyPrint
- std::string `to_string` ()
allows use of print on an expression
- std::string `to_pretty_string` ()
allows use of prettyPrint on an expression

Public Attributes

- std::string `name`
A string representing this [Variable](#) expression.

5.5.1 Detailed Description

[Variable](#) class: a derived expression class representing a variable expression.

More details here:

5.5.2 Constructor & Destructor Documentation

5.5.2.1 Variable()

```
Variable::Variable (
    std::string name)
```

[Variable](#) expression constructor.

FUNCTIONS

5.5.3 Member Function Documentation

5.5.3.1 equals()

```
bool Variable::equals (
    Expr * e) [virtual]
```

determines equality between two [Variable](#) expressions

Parameters

<code>e</code>	another expression that this expression is compared to
----------------	--

Returns

boolean, true for [Variable](#) expressions that are equal, otherwise false

Implements [Expr](#).

5.5.3.2 has_variable()

```
bool Variable::has_variable () [virtual]
```

determines if a variable expression contains a variable

Returns

boolean, always true as a variable is a variable

Implements [Expr](#).

5.5.3.3 interp()

```
int Variable::interp () [virtual]
```

interprets a [Variable](#) expression, evaluating the math contained within it

Returns

int an integer representing the result of the evaluated expression

Exceptions

e	throws an exception because a variable cannot be interpreted
-------------------	--

Implements [Expr](#).

5.5.3.4 print()

```
void Variable::print (
    std::ostream & os) [virtual]
```

prints this [Variable](#) expression to an outputstream

Parameters

os	an output stream for this Variable expression to be printed to
--------------------	--

Implements [Expr](#).

5.5.3.5 subst()

```
Expr * Variable::subst (
    std::string s,
    Expr * e) [virtual]
```

substitutes an expression for a given string

Parameters

<i>s</i>	a string in this expression to be replaced with <i>e</i>
<i>e</i>	an expression to substitute into this expression for string <i>s</i>

Returns

Expr* a new [Variable](#) expression with *e* substituted for occurrences of *s*

Implements [Expr](#).

5.5.4 Member Data Documentation

5.5.4.1 name

```
std::string Variable::name
```

A string representing this [Variable](#) expression.

MEMBER VARIABLES

The documentation for this class was generated from the following files:

- /Users/jasonablott/msdscript/expr.h
- /Users/jasonablott/msdscript/expr.cpp

Chapter 6

File Documentation

6.1 /Users/jasonablott/msdscript/cmdline.h

```
00001
00004
00005
00006 #ifndef MSDSCRIPT_H
00007 #define MSDSCRIPT_H
00008
00015 void use_arguments(int argc, char *argv[]);
00016
00017 #endif //MSDSCRIPT_H
```

6.2 /Users/jasonablott/msdscript/expr.h

```
00001
00004
00005 #include <string>
00006
00007 #ifndef EXPR_H
00008 #define EXPR_H
00009
00013 typedef enum {
00014     prec_none,           // = 0, for num or var
00015     prec_add,            // = 1, for addition expression
00016     prec_mult            // = 2, for multiplication expression
00017 } precedence_t;
00018
00028 class Expr {
00029 public:
00033     virtual ~Expr() = default;
00039     virtual bool equals(Expr *e) = 0;
00044     virtual int interp() = 0;
00049     virtual bool has_variable() = 0;
00056     virtual Expr* subst(std::string s, Expr* e) = 0;
00061     virtual void print(std::ostream& os) = 0;
00066     virtual void prettyPrint (std::ostream& os);
00072     virtual void prettyPrintAt (std::ostream& os, precedence_t precedence);
00077     std::string to_string();
00082     std::string to_pretty_string();
00083 };
00084
00085
00090 class Num : public Expr {
00091 public:
00095     int val;
00096
00100
00104     Num(int val);
00110     bool equals(Expr *e);
00115     int interp();
00120     bool has_variable();
00127     Expr* subst(std::string s, Expr* e);
00132     void print(std::ostream& os);
00133 };
00134
```

```
00135
00140 class Add : public Expr {
00141 public:
00145     Expr *lhs;
00146     Expr *rhs;
00147
00151
00155     Add(Expr *lhs, Expr *rhs);
00161     bool equals(Expr *e);
00166     int interp();
00171     bool has_variable();
00178     Expr* subst(std::string s, Expr* e);
00183     void print(std::ostream& os);
00188     void prettyPrint(std::ostream& os);
00194     void prettyPrintAt(std::ostream &os, precedence_t precedence);
00195 };
00196
00197
00202 class Mult : public Expr {
00203 public:
00207     Expr *lhs;
00208     Expr *rhs;
00209
00213
00217     Mult(Expr *lhs, Expr *rhs);
00223     bool equals(Expr *e);
00228     int interp();
00233     bool has_variable();
00240     Expr* subst(std::string s, Expr* e);
00245     void print(std::ostream& os);
00250     void prettyPrint(std::ostream& os);
00256     void prettyPrintAt(std::ostream &os, precedence_t precedence);
00257 };
00258
00259
00264 class Variable : public Expr {
00265 public:
00269     std::string name;
00270
00274
00278     Variable(std::string name);
00284     bool equals(Expr *e);
00290     int interp();
00295     bool has_variable();
00302     Expr* subst(std::string s, Expr* e);
00307     void print(std::ostream& os);
00308 };
00309
00310 #endif //EXPR_H
```

Index

Add, [9](#)
 Add, [10](#)
 equals, [10](#)
 has_variable, [10](#)
 interp, [11](#)
 lhs, [12](#)
 prettyPrint, [11](#)
 prettyPrintAt, [11](#)
 print, [11](#)
 subst, [12](#)

equals
 Add, [10](#)
 Expr, [13](#)
 Mult, [17](#)
 Num, [21](#)
 Variable, [24](#)

Expr, [12](#)
 equals, [13](#)
 has_variable, [13](#)
 interp, [14](#)
 prettyPrint, [14](#)
 prettyPrintAt, [14](#)
 print, [14](#)
 subst, [15](#)
 to_pretty_string, [15](#)
 to_string, [15](#)

has_variable
 Add, [10](#)
 Expr, [13](#)
 Mult, [17](#)
 Num, [22](#)
 Variable, [24](#)

interp
 Add, [11](#)
 Expr, [14](#)
 Mult, [17](#)
 Num, [22](#)
 Variable, [25](#)

lhs
 Add, [12](#)
 Mult, [19](#)

MSDScript, [1](#)

Mult, [16](#)
 equals, [17](#)
 has_variable, [17](#)
 interp, [17](#)

 lhs, [19](#)
 Mult, [17](#)
 prettyPrint, [18](#)
 prettyPrintAt, [18](#)
 print, [18](#)
 subst, [19](#)

name
 Variable, [26](#)

Num, [19](#)
 equals, [21](#)
 has_variable, [22](#)
 interp, [22](#)
 Num, [20](#)
 print, [22](#)
 subst, [22](#)
 val, [23](#)

prettyPrint
 Add, [11](#)
 Expr, [14](#)
 Mult, [18](#)

prettyPrintAt
 Add, [11](#)
 Expr, [14](#)
 Mult, [18](#)

print
 Add, [11](#)
 Expr, [14](#)
 Mult, [18](#)
 Num, [22](#)
 Variable, [25](#)

subst
 Add, [12](#)
 Expr, [15](#)
 Mult, [19](#)
 Num, [22](#)
 Variable, [25](#)

to_pretty_string
 Expr, [15](#)

to_string
 Expr, [15](#)

val
 Num, [23](#)

Variable, [23](#)
 equals, [24](#)
 has_variable, [24](#)
 interp, [25](#)

name, [26](#)
print, [25](#)
subst, [25](#)
Variable, [24](#)